

Python

José Luis Beltrán Márquez

ITESCA - CBTis 37

Cd. Obregón, Sonora; Enero 2025



ITESCA
Instituto Tecnológico
Superior de Cajeme



CBTis 37
José Ma. Leyva Pérez "Cajeme"

Índice

- 1 Introducción
- 2 Tipos de Datos
- 3 Secuencias
- 4 Control de Flujo
- 5 Funciones
- 6 Miscelanea



Introducción

Python fue creado a principios de los 90 por Guido van Rossum en CWI (<https://www.cwi.nl/>) en los Países Bajos como sucesor de un lenguaje llamado ABC. Guido sigue siendo el autor principal de Python, aunque incluye muchas contribuciones de otros.



Figura: Guido van Rossum

¿Porque Python?

Según el autor principal (Guido van Rossum), eligió el nombre "**Python**" porque es un gran fan de la película de comedia británica - 'Monty Python's Flying Circus'. Esto es lo que el autor declaró en una entrevista:

"Hace algunos años (en diciembre de 1989), estaba buscando un proyecto de programación que me mantuviera ocupado durante la semana de Navidad ... Elegí a Python como título de trabajo para el proyecto, ya que era un poco irreverente ... (Y un gran fan de Monty Python's Flying Circus) ".

Por cierto, el lenguaje se nombra después que la BBC mostrara "Monty Python's Flying Circus" y no tiene nada que ver con reptiles. Hacer referencias a Monty Python en la documentación no sólo está permitido, se anima!

The zen of Python -1 of 2-

Beautiful is better than ugly.

Explicit is better than implicit.

Simple is better than complex.

Complex is better than complicated.

Flat is better than nested.

Sparse is better than dense.

Readability counts.

Special cases aren't special enough to break the rules.

Although practicality beats purity.

Errors should never pass silently.

Unless explicitly silenced.

In the face of ambiguity, refuse the temptation to guess.



The zen of Python -2 of 2-

There should be one— and preferably only one —obvious way to do it
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.

Although never is often better than **right** now.

If the implementation is hard to explain, it's a bad idea.

If the implementation is easy to explain, it may be a good idea.

Namespaces are one honking great idea – let's do more of those!

El código de inspiración Python -1 de 2-

Bonito es mejor que feo

Explícito es mejor que implícito

Simple es mejor que complejo

Complejo es mejor que complicado

Plano es mejor que anidado

Disperso es mejor que denso

La legibilidad cuenta

Los casos especiales no son suficientemente especiales para romper las reglas

La practicidad bate a la pureza

Los errores no deben pasar inadvertidos

A menos que sean explícitamente silenciados



El código de inspiración Python -2 de 2-

En caso de ambigüedad, rechazar la tentación de adivinar
Debe haber una - y preferiblemente única- forma obvia de hacerlo
Aunque esa única forma no sea obvia en un primer momento, a no
ser que seas holandés

Ahora es mejor que nunca

Aunque nunca es mejor que **ahora** mismo

Si la implementación es difícil de explicar, es una mala idea

Si la implementación es fácil de explicar, debe ser una buena idea

Los espacios de nombres son una idea genial .. ¡hay que hacer más de esos!

[1]

Vocabulario

Las siguientes palabras, son palabras reservadas por Python y no pueden ser usadas por ti para nombrar cosas. Indican comandos al intérprete de Python:

and	del	from	not	while
as	elif	global	or	with
assert	else	if	pass	yield
break	except	import	nonlocal	class
lambda	in	raise	continue	finally
is	return	def	for	print
try	True	False	None	exec

Figura: Lista de Palabras Reservadas

El Intérprete y su entorno

Python es un lenguaje interpretado, que puede ahorrar un tiempo considerable durante el desarrollo del programa porque no es necesario la compilación y vinculación.

Puedes escribir las instrucciones en el intérprete, e ir ejecutando de una en una, si así lo deseas. Esa capacidad es conocida como programación interactiva. Sin embargo, la mayoría de los programas son mucho más largos, así que los escribimos en un archivo y decimos a python que ejecute los comandos en el archivo. En cierto sentido estamos dando a Python un guión llamado **script**, como convención, agregamos **.py** como el sufijo al final de estos archivos para indicar que contienen Python.

Tipos de Variables

Python tiene un set de tipos de variables inter-construidas, que pueden usarse como ladrillos para construir objetos más complejos, ejemplos: int, float, str, bool, complex list, dict, etc. Imagine un grupo de preguntas para una persona:

- ¿Cuántos hijos tiene?
- ¿Cuanto pesa en kg?
- ¿Cual es su nombre?
- ¿Es usted feliz?

¿De que tipo podrian ser las respuestas?

Asignación automática del tipo

Python no es tipado, lo que significa que no es necesario especificar el tipo de la variable, ya que la selección del tipo es automática.

¿Que pasa cuando escribes una asignación en python?

- Python calcula el lado derecho de la asignación.
- Guarda el resultado en una localidad de memoria.
- Asocia la localidad con la variable.
- Dependiendo del resultado de la operación a la derecha de la asignación, será el tipo de la variable.
- En caso de que la variable ya se estaba usando y era de otro tipo, Python hace la reasignación del tipo en forma automática.
- Si lo deseas, es posible especificar el tipo de la variable si usas el constructor del tipo.

Tipos de Datos

Tipo	Mutable	Inmutable	Secuencia	Collection
int		✓		
float		✓		
complex		✓		
bool		✓		
str		✓	✓	
tuple		✓	✓	✓
range		✓	✓	
list	✓		✓	✓
dict	✓			✓
set	✓			✓
personalizados	✓			

Identificadores válidos

```
abc
```

```
Abc
```

```
_x5
```

```
mi_x
```

```
mi_x1
```


Identificadores inválidos

2x
x.2
A-2
mi x
for

El signo '='

Es importante notar que en el contexto de Python, el signo `=` es usado para asignar un nombre a una variable. En otras palabras significa asociar un identificador con una variable u objeto.

X=2 no significa que X sea igual a 2, es simplemente que se toma el resultado de la derecha (que en este caso es 2), se guarda en una localidad de memoria, la cual es identificada por X.

Es mejor leer X es asociada a 2, de ésta manera puede ya tener sentido la siguiente sentencia: `X=X+1`, que en matemáticas definitivamente es extraña, pero en Python significa que el programa debe:

- Ir a la localidad de memoria asociada al identificador X
- Tomar el valor de la localidad y sumarle uno.
- Guardar el resultado en una localidad de memoria e identificarla por X (en este caso el mismo identificador)

int

A partir de Python 3, solamente existe un solo tipo de int (64 bits) que abarca a los anteriores int y long.

```
a = 5 + 2
print(a)
print(type(a))
print(f"El valor de a es {a} y su tipo es {type(a)}")
```

float

En python a diferencia de C, no hay distinción entre float y double. Todos son float, donde se emplea 1 bit para el signo, 11 bits para el exponente, y 52 bits para la mantisa.

```
a = 5 / 2
print(a)
print(type(a))
print(f"El valor de a es {a} y su tipo es {type(a)}")
```

str

Los str son secuencias de caracteres, pero son inmutables; es decir, una vez creados ya no se pueden cambiar.

```
a = "Hola mundo Python!"
print(a)
print(type(a))
print(f"El valor de a es {a} y su tipo es {type(a)}")

a = 'Hola mundo Python!'
print(a)
print(type(a))
print(f"El valor de a es {a} y su tipo es {type(a)}")

a = """
print(a)
print(type(a))
print(f"El valor de a es {a} y su tipo es {type(a)}")
```


complex

Se utilizan para representar números complejos, están formados por dos números de tipo float, la parte imaginaria lleva el identificador j.

```
a = 5 + 2j
print(a)
print(type(a))
print(f"El valor de a es {a} y su tipo es {type(a)}")
```

list

Una list es un contenedor **mutable** que contiene a otros objetos (donde cada objeto puede ser cualquier objeto, incluso otra list por ejemplo).

El orden de los objetos dentro de la list importa, en otras palabras, la list es una **secuencia** de objetos, no es una bolsa de objetos.

```
a = ["Hola", 5.6, 137, True]
print(a)
print(type(a))
print(f"El valor de a es {a} y su tipo es {type(a)}")
print(f"el elemento 0 es {a[0]}")
print(f"el elemento 1 es {a[1]}")
print(f"el elemento -1 es {a[-1]}")
print(f"el elemento -2 es {a[-2]}")
```


Accediendo a los elementos de la list

Se usan los corchetes cuadrados y el índice del elemento de la list:

- El indexado positivo es de izquierda a derecha, y comienza del 0,1,...,N-1
- El indexado negativo es de derecha a izquierda, y comienza del -1,-2,...-N

Indexado
Positivo

list

Indexado
Negativo

0	1	2	3
'Hola'	5.6	137	True
-4	-3	-2	-1

Figura: Indices en una lista



tuple

Es igual que las list con la diferencia que los datos son **inmutables**, es decir, una vez creados ya no se pueden cambiar. Para crear una tuple, se ponen todos los objetos separados por comas dentro de paréntesis, o bien, sólo separados por comas,

```
a = ("Hola", 5.6, 137, True)
print(a)
print(type(a))
print(f"El valor de a es {a} y su tipo es {type(a)}")
a = (4) #esto no es una tupla, ya que no tiene coma
print(a)
print(type(a))
print(f"El valor de a es {a} y su tipo es {type(a)}")
a = (4,)
print(a)
print(type(a))
print(f"El valor de a es {a} y su tipo es {type(a)}")
```

range

Los Rangos son **secuencias inmutables** de enteros, usados comúnmente en ciclos for.

Es un intervalo [inicio, fin) donde inicio es cerrado y fin es abierto

La sintaxis es **[inicio=0, fin, incremento=1)**

```
a = range(10)
print(a)
print(type(a))
print(f"El valor de a es {a} y su tipo es {type(a)}")
print (list(a)) #se convierte a list para poder imprimir en
                pantalla
print (list(range (0 ,10)))
print (list(range (0 ,10 , 2)))
print (list(range (0 , -10)))
print (list(range (0 , -10 , -1)))
print (list(range (-10, -20, -1)))
print (list(range (0)))
```

set

Los sets son collecciones desordenadas de objetos hashables (para ser hashables deben ser de tipo inmutable).

El set es una bolsa de objetos, por lo que los objetos dentro no pueden ser indexados del tipo de secuencia.

```
a = {5,2,8}
print(a)
print(type(a))
print(f"El valor de a es {a} y su tipo es {type(a)}")
a = set("528")
print(a)
print(type(a))
print(f"El valor de a es {a} y su tipo es {type(a)}")
```


operaciones con set -ejemplo-

```
universo={1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19}
impares={1,3,5,5,5,7,9,11,13,15,17,19}
multiplos_de_3={3,6,6,6,6,9,12,15,18}
multiplos_de_3_e_impares=impares.intersection(multiplos_de_3
)
multiplos_de_3_o_impares=impares.union(multiplos_de_3)
#pares=universo.symmetric_difference(impares)
#pares=impares.difference(universo)
pares=universo.difference(impares)
print("Universo:",universo)
print("Impares:",impares)
print("Pares:",pares)
print("Multiplos de 3:",multiplos_de_3)
print("Multiplos de 3 e impares:",multiplos_de_3_e_impares)
print("Multiplos de 3 o impares:",multiplos_de_3_o_impares)
```


dict

Para acceder a los elementos que están guardados en una lista, se usa el índice del elemento, ejemplo:

Indexado	0	1	2	3
Lista	'Hola'	5.6	137	True

Mientras que para acceder a los elementos que están guardados en un diccionario, se usa una llave asociada al elemento, ejemplo:

Indexado	Paola	Julian	Juan	Mireya
Diccionario	95	85	70	75

dict -ejemplos-

Los diccionarios guardan pares de datos (llave, valor), de forma tal que se necesita la llave para acceder al valor.

```
a = {} #diccionario vacio
a={"Paola":95,"Julian":85,"Juan":70,"Mireya":75}
print(a)
print(type(a))
print(f"El valor de a es {a} y su tipo es {type(a)}")
print("Calificacion de Juan ",a["Juan"])
print("Calificacion de Silvia ",a["Silvia"])#esto es un
    error, ponlo 1 linea abajo

a["Silvia"]=90 #inserta un elemento nuevo con llave silvia
print("a es ",a)
print("Juan" in a)
print("Daniel" in a)
del(a["Mireya"])
print("a es ",a)
print("Llaves ",a.keys())
print("Valores ",a.values())
```

None

None se refiere exactamente a la funcionalidad deseada: no es nada y no tiene ningún comportamiento.

El tipo **None** de Python es un objeto y se comporta como tal.

Como **None** es un objeto, no podemos usarlo para verificar si existe una variable. Es un valor ú objeto, no un operador que se usa para verificar una condición.

```
a = None
print(a)
print(type(a))
print(f"El valor de a es {a} y su tipo es {type(a)}")
if a:
    print("None es True")
elif a is False:
    print("None es False")
else:
    print("None es precisamente None..")
```

Desempacado de Secuencias

Podemos asignarle a varias variables los valores de una secuencia usando la siguiente sintaxis:

`[nomVar, ]...[*nomVar[, nomVar]...] = secuencia`

- Solo puede haber una variable estrellada.
- El número de variables no puede ser mayor que el tamaño de la secuencia.
- Si no hay una variable estrellada el número de las otras variables debe ser igual al tamaño de la secuencia.
- A cada variable no estrellada se le asigna un valor de la secuencia. A la variable estrellada se le asigna una lista con el resto de los valores de secuencia. Los valores se asignan en el orden de las variables.

Operaciones sobre Secuencias

`for e in seq` itera sobre los elementos de secuencia

`len(secuencia)` Regresa la longitud de la secuencia.

`min(secuencia)` Regresa el elemento mínimo de la secuencia.

`max(secuencia)` Regresa el elemento máximo de la secuencia.

`sorted(secuencia, key=None, reverse=False)` Regresa una secuencia con los elementos de secuencia ordenados. `secuencia` no se altera. El parámetro `key` establece el método de comparación. El parámetro `reverse` establece si el ordenamiento es en orden ascendente o descendente.

ésimo elemento de Secuencias

```
lista = [ -2 , -1 , 0 , 1 , 2]
cadena = "hola mundo python"
print(f"lista= {lista}")
print(f"lista[1]= {lista[1]}")
print(f"lista[-1]= {lista[-1]}")
print(f"cadena= {cadena}")
print(f"cadena[1]= {cadena[1]}")
print(f"cadena[-1]= {cadena[-1]}")
```


Concatenación de Secuencias (no rangos)

```
lista = [ -2 , -1 , 0 , 1 , 2]
lista2 = [4 , 5 , 6 , 7 , 8]
cadena = "hola mundo python "
cadena2 = "esplendido dia!"

print(f"listas concatenadas: {lista+lista2}")
print(f"listas concatenadas: {cadena+cadena2}")
```

Repetición de Secuencias (no rangos)

```
lista = [ -2 , -1 , 0 , 1 , 2]
cadena = "hola mundo python "

print(f"lista repetida: {3*lista}")
print(f"lista repetida: {3*cadena}")
```

Verificar pertenencia de elementos en Secuencias

```
lista = [ -2 , -1 , 0 , 1 , 2]

print(f"¿El 2 esta en la lista?: {2 in lista}")
print(f"¿El 5 esta en la lista?: {5 in lista}")
print(f"¿El 2 no esta en la lista?: {2 not in lista}")
print(f"¿El 5 no esta en la lista?: {5 not in lista}")
```

Iterar sobre los elementos de Secuencias

```
lista = [ -2 , -1 , 0 , 1 , 2]
cadena = "hola mundo python"

print(f"LISTA:")
for elemento in lista:
    print(f"elemento: {elemento}")

print(f"\nCADENA:")
for elemento in cadena:
    print(f"elemento: {elemento}")
```

mínimo, máximo y longitud de Secuencias

```
lista = [ -2 , -1 , 0 , 1 , 2]
cadena = "hola mundo python"

print(f"Mínimo de lista: {min(lista)}")
print(f"Máximo de lista: {max(lista)}")
print(f"Longitud de lista: {len(lista)}")

print(f"Mínimo de cadena: {min(cadena)}")
print(f"Máximo de cadena: {max(cadena)}")
print(f"Longitud de cadena: {len(cadena)}")
```

Ordenamiento de Secuencias

```
lista = [ -2 , -1 , 0 , 1 , 2]
cadena = "hola mundo python"

print (f"lista ascendente: {sorted (lista)}")
print (f"lista descendente: {sorted (lista , reverse = True
    )}")
print (f"funcion lamda: {sorted (lista , key= lambda n: n
    **2)}")
print (f"cadena ascendente: {sorted (cadena)}")
```

Operadores Aritméticos

Operación	operador	Ejemplo
Exponenciación	**	2**3
Positivo	+	+3
Negativo	-	-3
Multiplicación	*	2*3
División	/	3/2
Cociente	//	5//2
Residuo	%	5%2
Suma	+	5+2
Resta	-	5-2

Operadores Relacionales

Operación	operador	Ejemplo
menor	$<$	$2 < 3$
menor igual	$\leq$	$2 \leq 3$
mayor	$>$	$2 > 3$
mayor igual	$\geq$	$2 \geq 3$
igual	$==$	$3 == 2$
diferente	$!=$	$5 != 2$

Diferencia entre `==` y `is` en Python

Python ofrece el operador **`is`** para comparar identidad (si dos objetos son exactamente el mismo en memoria):

`==` Compara el valor de los objetos (igualdad).

`is` Compara la identidad de los objetos (si son la misma instancia).

Diferencia entre == y is en Python

```
# Comparación con ==
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b) # True (los valores son iguales)
print(a is b) # False (son diferentes objetos en memoria)

# Comparación con is
c = a
print(a is c) # True (son el mismo objeto en memoria)
```

Lógicos

Los operadores lógicos operan comparando los valores booleanos de sus operandos.

Al compararse, los siguientes valores de objetos son considerados como falsos: False, None, el cero en cualquier tipo numérico, cadenas o colecciones vacías. Los demás valores se consideran verdaderos.

Evaluación en Corto Circuito: A veces, para obtener el resultado de las operaciones and y or; al evaluar el operando izquierdo, ya no es necesario evaluar el operando derecho.

Operación	operador	Ejemplo
not	not	not (A<0)
and	and	(A>0) and (A<100)
or	or	(A>0) or (A<100)

Membresía

Operación	operador	Ejemplo
¿un valor existe en una secuencia?	in	5 in [2,5,3,7]
¿un valor no existe en una secuencia?	not in	5 not in [2,5,3,7]

Precedencia

Si una expresión contiene más de un operador, el orden en que se efectúan las operaciones está dado por las siguientes reglas:

- La precedencia de operadores determina el orden en que se evalúan las expresiones.
- Los operadores con mayor precedencia se evalúan primero.
- Se puede usar paréntesis () para alterar el orden de evaluación.
- Las expresiones encerradas entre paréntesis son evaluadas primero.
- Si en una expresión hay operadores de igual precedencia, las operaciones se agrupan de izquierda a derecha, excepto por la exponenciación que se agrupa de derecha a izquierda.

Orden de precedencia de operadores en Python

Precedencia	Operadores
Más alta	() (paréntesis)
2	** (potencia)
3	+x, -x, ~ (positivo, negativo, complemento)
4	*, /, //, % (multiplicación, división, división entera, módulo)
5	+, - (suma y resta)
6	<<, >> (desplazamiento de bits)
7	& (AND bit a bit)
8	^ (XOR bit a bit)
9	(OR bit a bit)
10	in, not in, is, is not, <, <=, >, >=, !=, == (comparaciones)
11	not (negación lógica)
12	and (AND lógico)
Más baja	or (OR lógico)

Ejemplo 1: Operaciones Matemáticas

```
resultado = 2 + 3 * 4
print(resultado)  # Salida: 14 (multiplicación antes
                  de suma)

resultado = (2 + 3) * 4
print(resultado)  # Salida: 20 (paréntesis alteran
                  el orden)
```

Ejemplo 2: Operadores Lógicos

```
a = True
```

```
b = False
```

```
c = True
```

```
resultado = a or b and c
```

```
print(resultado)  # Salida: True (AND se evalúa  
                  antes que OR)
```

```
resultado = (a or b) and c
```

```
print(resultado)  # Salida: True (paréntesis alteran  
                  el orden)
```


Ejemplo 3: Comparaciones y Operadores Bit a Bit

```
resultado = 5 > 3 and 3 < 2 or 4 == 4
print(resultado)  # Salida: True
```

```
resultado = (5 > 3) and (3 < 2) or (4 == 4)
print(resultado)  # Salida: True (misma evaluación
                  con paréntesis)
```

```
resultado = 5 & 3 | 2
print(resultado)  # Salida: 3 (AND bit a bit antes
                  que OR bit a bit)
```

Estructuras de Control - Condicionales

Python utiliza las palabras clave `if`, `elif` y `else` para manejar decisiones.

Ejemplo:

```
# Verificar si un número es positivo, negativo o  
cero  
numero = -5  
  
if numero > 0:  
    print("El número es positivo")  
elif numero < 0:  
    print("El número es negativo")  
else:  
    print("El número es cero")
```

Estructuras de Control - for

Los bucles for se usan para iterar sobre colecciones.

Ejemplo con for:

```
# Imprimir números del 1 al 5  
for i in range(1, 6):  
    print(i)
```

Estructuras de Control - break en ciclos for

La sentencia break termina inmediatamente el ciclo, pero no se ejecuta el else. Si existen ciclos anidados, solo termina el ciclo más interno al que pertenece el break.

```
for char in "computadora":  
    print(char)  
print("Fin del ciclo 1")  
  
for char in "computadora":  
    if char == 'p':  
        break  
    print(char)  
print("Fin del ciclo 2 (break)")
```

Estructuras de Control - continue en ciclos for

La sentencia continue es usada para saltar el resto del código de la actual iteración del ciclo. Sin embargo, el ciclo no termina y continua la siguiente iteración.

```
for char in "computadora":  
    print(char)  
print("Fin del ciclo 1")  
  
for char in "computadora":  
    if char == 'p':  
        continue  
    print(char)  
print("Fin del ciclo 2 (continue)")
```

Estructuras de Control - enumerate en ciclos for

Para tener un contador de los ciclos en for

```
citricos = ["naranja", "limon", "toronja", "mandarina",  
            , "sidra", "lima"]  
  
for index,item in enumerate(citricos):  
    print(f"indice: {index}, producto: {item}")
```

Estructuras de Control - while

Los bucles while se usan para realizar repeticiones.

Ejemplo con while:

```
# Contar hacia atrás desde 5
contador = 5
while contador > 0:
    print(contador)
    contador -= 1
```

¿Qué es una excepción?

En Python, una excepción es un evento que ocurre durante la ejecución de un programa y que interrumpe el flujo normal de instrucciones.

- Las excepciones ocurren por errores lógicos, de código o por problemas inesperados.
- El manejo adecuado de excepciones es clave para evitar que un programa falle abruptamente.

Sintaxis de try-except

La estructura básica para manejar excepciones en Python es:

```
try:
    # Bloque de código que puede generar una excepción
except TipoDeExcepcion:
    # Bloque de código para manejar la excepción
finally:
    # (Opcional) Bloque de código que se ejecuta siempre
```

Ejemplo:

```
try:
    num = int(input("Introduce un número: "))
    print("El número es:", num)
except ValueError:
    print("Error: Debes introducir un número válido.")
    "
```

Manejo de múltiples excepciones

Puedes manejar diferentes tipos de excepciones en un bloque try usando varios bloques except.

Ejemplo:

```
try:
    resultado = 10 / int(input("Introduce un divisor
    : "))
    print("Resultado:", resultado)
except ZeroDivisionError:
    print("Error: No puedes dividir entre cero.")
except ValueError:
    print("Error: Debes introducir un número válido.
    ")
```

Bloque finally

El bloque `finally` se ejecuta siempre, independientemente de si se produce una excepción o no.

Ejemplo:

```
try:
    archivo = open("datos.txt", "r")
    contenido = archivo.read()
    print(contenido)
except FileNotFoundError:
    print("Error: El archivo no existe.")
finally:
    print("Cerrando el archivo.")
    archivo.close()
```

Ventajas del manejo de excepciones

- Permite gestionar errores de forma controlada.
- Mejora la robustez y la estabilidad del programa.
- Facilita la depuración al proporcionar información específica sobre los errores.
- Garantiza que se realicen tareas críticas como liberar recursos o cerrar archivos.

Buenas prácticas al manejar excepciones

- Especifica el tipo de excepción que desees manejar (no uses `except: genérico`).
- Proporciona mensajes de error claros y útiles.
- Evita el uso excesivo de excepciones para controlar flujos lógicos normales.
- Siempre utiliza `finally` o el administrador de contextos `with` para garantizar la liberación de recursos.
- El manejo de excepciones es esencial para crear programas robustos y fáciles de mantener.
- Utilizar correctamente `try`, `except`, y `finally` ayuda a prevenir fallos inesperados.
- Adoptar buenas prácticas asegura que tu código sea más legible y confiable.

¿Qué es el administrador de contexto with?

El administrador de contexto `with` en Python es una construcción que permite:

- Gestionar recursos de manera eficiente y segura.
- Asegurar que ciertos procedimientos, como cerrar archivos o liberar recursos, se realicen correctamente.
- Reducir errores y simplificar el código.

Sintaxis del bloque with

La sintaxis básica de with es:

```
with expresion as variable:  
    # Bloque de código
```

Ejemplo: Manejo de archivos

```
with open("archivo.txt", "r") as archivo:  
    contenido = archivo.read()  
    print(contenido)  
# El archivo se cierra automáticamente al salir del  
    bloque
```

Ventajas de with

- **Gestión automática de recursos:** Libera recursos como archivos abiertos o bloqueos sin necesidad de hacerlo manualmente.
- **Evita fugas de recursos:** Garantiza que los recursos se liberen incluso si ocurre una excepción.
- **Código más limpio:** Reduce la necesidad de bloques `try-finally` explícitos.

Funciones

Las funciones permiten reutilizar código. Se definen con la palabra clave `def`.

Ejemplo:

```
# Definir una función que suma dos números
def suma(a, b):
    return a + b

# Llamar a la función
resultado = suma(3, 5)
print("La suma es:", resultado)
```

Documentación de una función

Python tiene una ingeniosa manera de documentación, a la que se conoce como docstrings.

DocStrings es una herramienta importante que debe utilizarse, ya que ayuda a documentar mejor el programa y facilitar su comprensión.

Sorprendentemente, podemos incluso obtener la docstring de, por ejemplo, una función cuando el programa está en ejecución!

Un comentario en la primera línea de una función, es la docstring para esa función. La convención seguida para una docstring es un comentario de varias líneas donde la primera línea comienza con mayúscula y termina con un punto. A continuación, la segunda línea está en blanco seguida de cualquier explicación a partir de la tercera línea. Se recomienda encarecidamente seguir esta convención.

Podemos acceder a la docstring de una función usando el atributo `__doc__` (es un doble subrayado)

Si has utilizado `help()` en Python, entonces ya has visto el uso de docstrings!, lo que hace es sólo buscar el atributo `__doc__` de esa función.

Espacios de Nombres (namespace)

- En Python, cada módulo o cada función o cada clase tiene su propio espacio de nombres.
- Este espacio de nombres se llama: espacio de nombres local, que básicamente significa que es local a esa función en particular.
- Cuando se llama una función e inicia su ejecución, un espacio de nombres se crea para la función, y cuando la función termina, su espacio de nombres se elimina.
- Las variables que se definen en el cuerpo principal de un programa se denominan variables globales. El espacio de nombres global contiene todas las variables globales.



Espacios de Nombres

```
def fun1(x,a):
    y=x+a
    return y
```

#Main Programa

```
x=3
```

```
z=x+12
```

```
a=fun1(z,9)
```

```
x=a+7
```

spacename Global		spacename fun1	
fun1	fun1(x,a)	x	15
x	3→31	a	9
z	15	y	24
a	24	retorno	24

Espacios de Nombres

Usando la variable local:

```
def fun1():  
    x=7  
    print("x de fun1",x)  
  
#Main Programa  
x=15 #variable Global  
fun1()  
print("x de Main",x)
```

Usando la variable global:

```
def fun1():  
    print("x de fun1",x)  
  
#Main Programa  
x=15 #variable Global  
fun1()  
print("x de Main",x)
```

Espacios de Nombres

Usando la variable padre:

```
def fun1():
    def fun2():
        print("x de fun2",x)
    )
    x=7
    print("x de fun1",x)
    fun2()
```

#Main Programa

x=15 #variable Global

fun1()

print("x de Main",x)

Usando la variable global:

```
def fun1():
    def fun2():
        global x
        print("x de fun2",x)
    )
    x=7
    print("x de fun1",x)
    fun2()
```

#Main Programa

x=15 #variable Global

fun1()

print("x de Main",x)

Funciones como objetos

- Las funciones son objetos del tipo function
- Una función puede asignársele a una variable
- El nombre de la variable puede usarse para invocar a la función

Ejemplo:

```
def mayor(x, y):  
    if (x > y):  
        return x  
    return y  
  
print(type(mayor))  
may = mayor  
print(may(2, 7))
```


Funciones como Argumentos de Funciones

- Una función puede recibir como argumento otra función
- Dentro de la primera función, el argumento puede usarse para llamar a la segunda función



Funciones como Argumentos de Funciones

```
def ordenar_estudiantes_por_nombre(estudiante):
    return estudiante[0]
def ordenar_estudiantes_por_matricula(estudiante):
    return estudiante[1]
def ordenar_estudiantes_por_calificacion(estudiante):
    return estudiante[2]
estudiantes = [
    ('Juan', '01', 8.4),
    ('Zenía', '46', 7.5),
    ('Belinda', '24', 9.5)]
print('sin Ordenar:', estudiantes)
estudiantesOrdenados= sorted(estudiantes, key=
    ordenar_estudiantes_por_nombre)
print('por Nombre:', estudiantesOrdenados)
estudiantesOrdenados= sorted(estudiantes, key=
    ordenar_estudiantes_por_matricula)
print('por Matricula:', estudiantesOrdenados)
estudiantesOrdenados= sorted(estudiantes, key=
    ordenar_estudiantes_por_calificacion)
print('por Calificacion:', estudiantesOrdenados)
```

¿Qué es una función lambda?

En Python, una función **lambda** es una función anónima que puede definirse en una sola línea de código.

- Se utiliza para definir funciones simples de forma rápida.
- No requiere la palabra clave `def` ni un nombre.
- Puede tomar cualquier número de argumentos, pero solo una expresión.
- Las funciones lambda son una herramienta poderosa en Python para simplificar tareas repetitivas.
- Es importante usarlas cuando la simplicidad sea una prioridad y evitar su abuso en casos donde la legibilidad sea clave.
- Al combinarlas con funciones como `map`, `filter` y `reduce`, se puede escribir código conciso y eficiente.

Sintaxis de una función lambda

La sintaxis de una función lambda es:

```
lambda argumentos : expresion
```

Ejemplo:

```
# Función lambda que suma dos números  
suma = lambda x, y: x + y  
  
# Usar la función lambda  
print(suma(5, 3)) # Salida: 8
```

Función map

La sintaxis de una función map es:

```
map(función, colección)
```

función Una función que define cómo transformar los elementos

colección Una colección (como una lista o tupla) a la que se aplica la función

Función filter

La sintaxis de una función map es:

```
filter(función, colección)
```

función Una función que devuelve True o False para cada elemento

colección Una colección cuyos elementos se filtrarán según la función



Usos comunes de las funciones lambda

Ejemplo con map:

```
# Elevar al cuadrado cada número en una lista
numeros = [1, 2, 3, 4]
cuadrados = list(map(lambda x: x**2, numeros))
print(cuadrados)  # Salida: [1, 4, 9, 16]
```

Ejemplo con filter:

```
# Filtrar números mayores que 2
numeros = [1, 2, 3, 4]
mayores = list(filter(lambda x: x > 2, numeros))
print(mayores)  # Salida: [3, 4]
```

Limitaciones de las funciones lambda

- Simplifican el código al no requerir la definición de una función completa.
- Útiles para operaciones rápidas y temporales.
- Se pueden usar como argumentos en funciones de orden superior.
- Solo pueden contener una expresión.
- Difícil de depurar debido a la falta de un nombre descriptivo.
- Menor legibilidad en casos complejos.

Módulos y Librerías

Python tiene una gran cantidad de librerías disponibles. Puedes importarlas usando **import**

Ejemplo:

```
import math
# Calcular la raíz cuadrada de un número
numero = 16
raiz_cuadrada = math.sqrt(numero)
print("La raíz cuadrada es:", raiz_cuadrada)
```

¿Qué son los módulos y librerías?

- Un **módulo** es un archivo de Python que contiene definiciones y código reutilizable, como funciones, clases o variables.
- Una **librería** es una colección de módulos empaquetados que ofrecen funcionalidades predefinidas.
- Los módulos y librerías ayudan a estructurar el código y evitar la duplicación.



Trabajando con módulos

Un módulo es simplemente un archivo `.py` que puedes importar.

Ejemplo: Crear un módulo llamado `mimodulo.py`

```
# Contenido de mimodulo.py
def saludar(nombre):
    return f"Hola, {nombre}!"
```

Importar y usar el módulo:

```
import mimodulo

mensaje = mimodulo.saludar("Carlos")
print(mensaje)  # Salida: Hola, Carlos!
```


Ejemplo: Librería random (Números Aleatorios)

```
import requests

respuesta = requests.get("https://api.github.com")
print(respuesta.status_code)    # Salida: 200
print(respuesta.json())         # Imprime el contenido
                                en formato JSON
```

```
import random

print(random.randint(1, 10))    # Número aleatorio
                                entre 1 y 10
print(random.choice(['rojo', 'azul', 'verde'])) #
                                Color aleatorio
```

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ≡ ≡ ↺ 🔍 ↻

Buenas Prácticas al Usar Módulos y Librerías

- Usa nombres descriptivos para tus módulos y funciones.
- Evita importar todo con `from module import *`.
- Mantén tus librerías actualizadas con `pip`.
- Documenta tus módulos para facilitar su uso.



Formato de str

Cuando anteponeamos la **f** a un **str**, estamos diciendo que es un str formateado

```
var1 = 123.456
print(f"var1 = {var1}")
```

y definitivamente podemos formatear la salida, dándole un ancho específico, cambiando los dígitos de precisión u otro formato. por ejemplo, definir solo 1 dígito de precisión.

```
var1 = 123.456
print(f"var1 = {var1:.1f}")
```


Especificador de formato -: , relleno-

Un especificador de formato tiene la siguiente sintaxis:

```
: [relleno]alineacion [signo] [#] [0] [ancho] [,] [.precision] tipo
```

Descripción del especificador de formato:

(:) Todo especificador de conversión empieza con el carácter :

relleno Esta opción aplica solo en el caso de que se especifique una alineación. Especifica el carácter con que se rellenarán los espacios en blanco dentro del campo. Puede ser cualquier carácter excepto " o '. El valor por omisión es ' '.

Especificador de formato -alineación-

alineación Especifica como se alineará el valor dentro del campo.

- (**<**) El valor se mostrará justificado a la izquierda. Este es el valor por omisión con la mayoría de los objetos.
- (**>**) El valor se mostrará justificado a la derecha. Este es el valor por omisión de los valores numéricos.
- (**=**) Solo aplica a los valores numéricos. Coloca los caracteres de relleno después del signo pero antes de los dígitos.
- (**^**) Centra el valor dentro del campo.

Especificador de formato -signo-

signo Aplica solo a los tipos numéricos. Especifica cómo aparecerá el signo.

- (-) El signo sólo se mostrará en los números negativos. Este es el valor por omisión.
- () (Un espacio), Deberá dejarse un espacio como prefijo de un número positivo.
- (+) El valor aparecerá con el prefijo '+' o '-' dependiendo de su signo.

Especificador de formato -#-

- (#) Si se utiliza la opción #, los valores se formatearán usando el formato alterno de acuerdo a lo siguiente.
 - (b) Se agrega el prefijo 'Ob' al número
 - (o) Se agrega el prefijo 'Oo' al número.
 - (x, X) Se agrega el prefijo 0x o 0X al número.
 - (e, E, f, F) El resultado tendrá el punto decimal aún si no hay dígitos después del punto.
 - (g, G) El resultado tendrá el punto decimal y conservará los ceros después de él.

Especificador de formato -tipo int-

tipo int Es el carácter de conversión de tipo. Especifica como se mostrarán los datos

- (s) String
- (b) binario
- (c) caracter
- (d) decimal
- (o) octal
- (x,X) hexadecimal
- nada decimal

Especificador de formato -tipo float-

tipo float Es el carácter de conversión de tipo. Especifica como se mostrarán los datos

- (e,E) Flotante en notación científica. La precisión por omisión es 6.
- (f, F) Flotante en notación de punto fija. La precisión por omisión es 6.
- (g,G) Flotante en notación científica si el exponente es menor que -4 o no menor que la precisión, en notación de punto fijo en caso contrario.
- (%) Porcentaje. Multiplica el valor por 100 y despliega u valor en notación de punto fijo, seguido del carácter %.

nada decimal



Especificador de formato

```

num1 = 123
num2 = -123
num3 = 123456789
num4 = 0.123456789
str1 = "abc"

#{[nombre del campo][:[[relleno]alineacion][signo][#][0][ancho][,][.precision]
tipo]}

print("{0} y {1}".format("carne", "huevos"))
print("{1} y {0}".format("carne", "huevos"))
print("Esta {comida} es {adjetivo}.".format(comida="carne", adjetivo="espantosa"
))

print(f"{str1:-<10} {num1:++<10}") #relleno
print(f"{str1:<10} {num1:<10}") #Alineacion y ancho
print(f"{str1:>10} {num1:<10}") #Alineacion y ancho
print(f"{num1:-} {num1: } {num1:++}") #signo
print(f"{num2:-} {num2: } {num2:++}") #signo
print(f"{num1:#b} {num1:#o} {num1:#x} {num1:#e} {num1:#g}") ##
print(f"{num2:010} {num2:05} {num2:0}") #0

for x in range(1,11):
    print(f"{x:2} {x*x:3} {x*x*x:4}") #ancho

print(f"{num2:,,} {num3:,,} {num4:,,}") #,
print(f"{num4:} {num4:.2} {num4:.4}") #precision
print(f"{num3:d} {num3:f} {num3:g}") #tipo

```


Entrada

Para leer una entrada de usuario se usa la función **input()** que tiene la siguiente sintaxis: `str input([prompt])`
ejemplos:

```
nombre_usuario=input("Cual es tu nombre:")  
print("Un gusto conocerte ",nombre_usuario)
```

El argumento *prompt* es opcional y lo siguiente también funciona, aunque el usuario puede no saber que es lo que quiere el programa:

```
nombre_usuario=input()  
print("Un gusto conocerte ",nombre_usuario)
```

Entrada

La función **input()** siempre regresa un *str*, aunque el usuario escriba un número. Hay que tener cuidado con ese detalle, ejemplo:

```
centigrados= input("centigrados? ")
fahrenheit= (centigrados*9/5)+32
print(fahrenheit)
```

Nos da un error, ya que la variable *centigrados* es de tipo *str*.
¿Que tenemos que hacer?, Respuesta: Convertir la variable *str* a *float*.

```
centigrados= float(input("centigrados? "))
fahrenheit= (centigrados*9/5)+32
print(fahrenheit)
```


Referencias I

- [1] Arturo Fernández Montoro. *Python 3 al descubierto*. Spanish. OCLC: 946343802. México: Alfaomega, 2013. ISBN: 978-607-707-718-3.
- [2] Guido Van Rossum. *Python/C API Reference Manual*. Jun. de 2001. URL: <https://docs.python.org/2.0/api/api.html> (visitado 24-05-2017).